

Duckietown Report

Conditional Imitation Learning with VLM-Guided Behavior Planning for Robust Control

DuckieExpress

Elia Reppchen, elia.reppchen@student.uni-tuebingen.de, 2026

Eric Berger, eric.berger@student.uni-tuebingen.de, 2026

Felix Schlechte, felix.schlechte@student.uni-tuebingen.de, 2026

Pankaj Bora, pankaj.bora@student.uni-tuebingen.de, 2026

Abstract

While implementing Duckietown Learning Experiences (LXs) we experienced the limitations of classical modular pipelines first hand. We overcome these limitations by training conditional imitation learning models on segmented images that drive the Duckiebot according to a per-frame intent, produced by a fine-tuned VLM-guided planner following a high-level plan. With this setup the bot robustly performs lane-following and turning at intersections, enabling a safe driving experience in Duckietown.

1. Introduction

Duckietown is an educational and research platform that enables the exploration of autonomous driving concepts using a simple and low-cost environment. It also provides Learning Experiences (LXs) in which we learned about basic concepts of self-driving cars. However, a number of problems arose during the LXs that made it difficult or even impossible to control the Duckiebot reliably. Most important, our Duckiebot shows **strong drift**, likely due to hardware issues. Because the provided odometry algorithms rely on correct wheel information we were not able to control our Duckiebot with a PID controller. Another problem we encountered was in the Computer Vision LX. We were supposed to find lower- and upper-bounds of HSV values by hand to correctly segment lanes in the images that are recorded by the bot. However, we were not able to find good values that are **robust under various lighting conditions**. As a consequence, the lane following module did not work well. In this report we address these challenges by using a two stage pipeline: first **segmenting input frames** before subsequently doing **conditional imitation learning (CIL)**. We find that this pipeline enables reliable control of our Duckiebot. In addition to that, we provide a high-level planner which parses simple text instructions into a plan (a queue of maneuvers). A finite-state machine (FSM) based behavior planner, guided by a fine-tuned Vision Language Model (VLM), steps through the queue to produce the intent needed to condition our CIL controller.

2. Method

To overcome the issues outlined in Chapter 1, we control our Duckiebot using neural networks. We solve ambiguous situations like intersections by *conditioning* our model on *high-level commands* or *intents* c_t . In the following, we describe how we collected the data, what network architectures we used, and how we trained them. A high level overview of our pipeline can be viewed in Figure A.1.

2.1. Data Collection

To collect the imitation learning dataset, we built a custom data collection node with a keyboard-controlled UI. The keys 'w', 'a', 's', 'd' drive the bot: 'w'/'s' set forward/backward velocity, and holding 'a' or 'd' adds an angular-velocity offset ω_a or ω_d to steer left or right; these offsets and the base velocities are hyperparameters, tunable live from the UI. The keys 'i', 'j', 'l', 'k' set the current intent to straight, left, right, or lane-following at intersections. For each frame we log the image f_t , wheel velocities $v_t = [v_t^l, v_t^r]^T$, the intent c_t , and the currently pressed keys, producing 10809 datapoints from about 20 minutes of driving.

2.2. YOLO segmentation model

We trained a YOLO26 [JQL⁺26] semantic segmentation model by using the ultralytics python library. We used the nano variant which has 1.6M parameters. Concretely, we train the model to predict 3 semantic classes: the white lane markings, the dashed yellow lane markings and the red markings. To train the YOLO model we generated high-quality masks with SAM3 [CGH⁺26] from our imitation learning dataset. These masks serve as the ground-truth labels when training the YOLO model. During training we used the pretrained yolo26n-sem checkpoint from the ultralytics library with default settings, only adjusting the input image size, and trained for 5 epochs. The resulting YOLO model produces very good segmentations of the lane markings in Duckietown. See Figure A.2 for an example segmentation mask.

2.3. Conditional Imitation Learning

We tried multiple ways to condition a neural network on high-level commands. In the following, we describe the two most successful approaches.

2.3.1. Multiple Head Architecture (MH)

This architecture is based on [CML⁺18]. As visualized in Figure A.6, we first pass the current frame f_t to a CNN backbone to produce visual features. Based on the current intent, we

then pass these features to a specialized fully-connected head which then predicts the wheel velocities.

2.3.2. Feature-wise Linear Modulation (FiLM)

The second model we train also uses a convolutional backbone with subsequent linear layers to predict the wheel velocities given a segmented input frame. It relies on the FiLM [PSdV⁺17] conditioning mechanism for the high-level commands. Given a feature map x_{in} a FiLM layer learns a shift coefficient β and scale coefficient γ that augment the input $x_{\text{out}} = \gamma \odot x_{\text{in}} + \beta$. For a convolutional layer this is done at the channel dimension. We implement an encoder that takes as input the one-hot encoded intent and projects to a shared embedding. For each layer that is augmented with FiLM we have a separate projection that outputs the correct dimensionality. All FiLM layers are initialized with $\gamma = 1$ and $\beta = 0$. A detailed overview of the FiLM-conditioned neural network can be found in Figure A.7.

2.3.3. Model Training

We train both models end-to-end using a 80/20 train/test split and minimize the MSE-loss $\mathcal{L}(\theta) = \frac{1}{|\mathcal{B}|} \sum_{(s_t, c_t, v_t) \in \mathcal{B}} (f_\theta(s_t, c_t) - v_t)^2$, where $\mathcal{B}$ is a batch with batchsize $|\mathcal{B}| = 64$ and f_θ is the respective neural network. We use Adam as an optimizer. Since the intents are not equally distributed in the dataset ('lane following' $\approx 73\%$, 'left' $\approx 11\%$, 'straight' $\approx 10\%$, 'right' $\approx 6\%$), we oversample the 'left', 'right', and 'straight' intents. We define a sampling strategy as an array of percentages with the order [lane following, straight, left, right]. For our first baseline approach, we use the sampling strategy [40, 20, 20, 20]. We use the model with the lowest validation loss.

2.4. VLM-Guided Behavior Planning

The intent c_t used to condition our controller (Section 2.3) is produced by a behavior planner. Given a text instruction (e.g. "take the second left, then a right and stop"), we parse it into a plan: a queue of maneuvers to execute in order. A finite-state machine (FSM, Figure A.3) consumes this queue one maneuver at a time. While lane-following it asks a VLM, once per frame, whether a junction is ahead and whether the target direction is open; once this is confirmed for a few consecutive frames it emits the corresponding intent, waits for the VLM to confirm the junction has been cleared, then advances to the next maneuver. The VLM is a visual oracle: a per-frame classifier that grounds the FSM's transitions in the current image. We initially queried a general-purpose VLM zero-shot, but found this too unreliable to drive the FSM, particularly on the exit-cleared question (Appendix A.5). We therefore fine-tune a classification head on top of a frozen SmolVLM2 [MZF⁺25] vision backbone to answer the FSM's two questions directly, and compare it against zero-shot, few-shot, and a CNN-only baseline in Appendix A.5.

2.5. ROS Nodes and Inference

The pipeline runs as four ROS nodes, all subscribing to `camera_node/image/compressed`. **Data collection** splits into two nodes. A teleoperation UI publishes `Twist2DStamped` on `car_cmd_switch_node/cmd` off a constant 10 Hz timer. Duckietown’s kinematics node converts v/ω into per-wheel velocities on `wheels_driver_node/wheels_cmd`. A logger node time-syncs the camera and wheel-velocity topics (0.5 s slop) and writes one row per pair while recording, matching Section 2.1. The **CIL inference node** crops each frame, segments it with YOLO, resizes and colorizes the mask, and feeds it with the one-hot intent into CIL (MH or FiLM, Section 2.3), loaded once at startup. Publishing is coupled to inference: a `WheelsCmdStamped` goes out only once a frame is processed, capped at 12 FPS. If the gap since the last publish exceeds $2\times$ that interval we log a warning. The wheels driver holds the last command until a new one arrives. Intent normally comes from `data_collector/intent`, the topic the teleoperation node publishes to. The **VLM inference node** (Section 2.4) adds a thread running the FSM and oracle on top of the CIL node, reusing its segmentation output so YOLO only runs once per frame. Each cycle the FSM queries the oracle, updates its state, and overwrites the intent, which the controller picks up on the next frame.

3. Experiments

In this section we test our CIL models on specific tasks and ablate training choices. Experiments comparing the F1-perormance of different VLM-oracle strategies can be found in Appendix A.5. To test our CIL models we trained a baseline model for both the MH and the FiLM architecture to compare between these two approaches. We refer to them as **a**) in Table 3.1. We observed, that both models have specific weaknesses that may be due to underrepresentation in our oversampling strategy. Therefore we tried another oversampling strategy, which we will describe in the results Section 3.1. In Table 3.1, we will refer to these two models as **b**).

3.1. CIL Results

The exact task definitions can be observed in Fig. A.4. We have tasks that test each intent in isolation and also do a long run to test long term stability. For each task, we test our model five times in each direction of travel and report how often the model succeeds. The results can be viewed in Table 3.1. We observe that in general, both architectures are able to control our bot reliably, but have some specific weaknesses or biases. For example the MH architecture sometimes fails on the isolated intent tasks, especially on left turns at intersections, but is more robust at the long-term experiment, because it is able to recover from slight errors. FiLM on the other hand is basically perfect on the isolated intent tasks (not as accurate on

Table 3.1.: Success rates for the evaluated tasks on five runs each.

Task	Multiple Heads		FiLM	
	a)	b)	a)	b)
Semicircle (left/right) (Fig. A.4a)	5/4	4/5	5/5	5/5
Long corner (left/right) (Fig. A.4b)	5/5	5/4	5/5	5/5
Straight at intersection (4-way/3-way) (Fig. A.4)	5/5	5/5	5/5	5/5
Left turn at intersection (4-way/3-way) (Fig. A.4)	4/2	5/5	5/5	5/5
Right turn at intersection (4-way/3-way) (Fig. A.4)	5/5	5/5	5/5	5/5
Long run (Fig. A.5)	5	5	4	5

straight intent), but surprisingly is not as robust on the long term task. Concerning **a)** we find, that FiLM is in general a bit more accurate. To improve the weaknesses of our models, we retrained our models with an adjusted sampling strategy. For MH we used [30, 20, 30, 20] and for FiLM [30, 30, 20, 20]. So **b)**, we observe, that this indeed helped to fix weaknesses. Especially the MH model improved the performance on the left turns severely but slightly loses performance on lane following. Another unintentional and interesting observation is, that both models are robust under wrong intent setting. For example given the right intent, the MH model can also do lane following as long it has no left corners, enabling it to run in a right turn loop.

4. Conclusion

In conclusion, we demonstrate that our Conditional Imitation Learning models are able to navigate robustly through Duckietown while following commands provided by our VLM-based planner. Through our experiments we find that both MH and FiLM based CIL models are able to produce good driving policies and that the data sampling strategy matters. We believe our pipeline is a strong foundation which can be extended to handle more complex scenarios and tasks in Duckietown.

A. Appendix

A.1. Figures

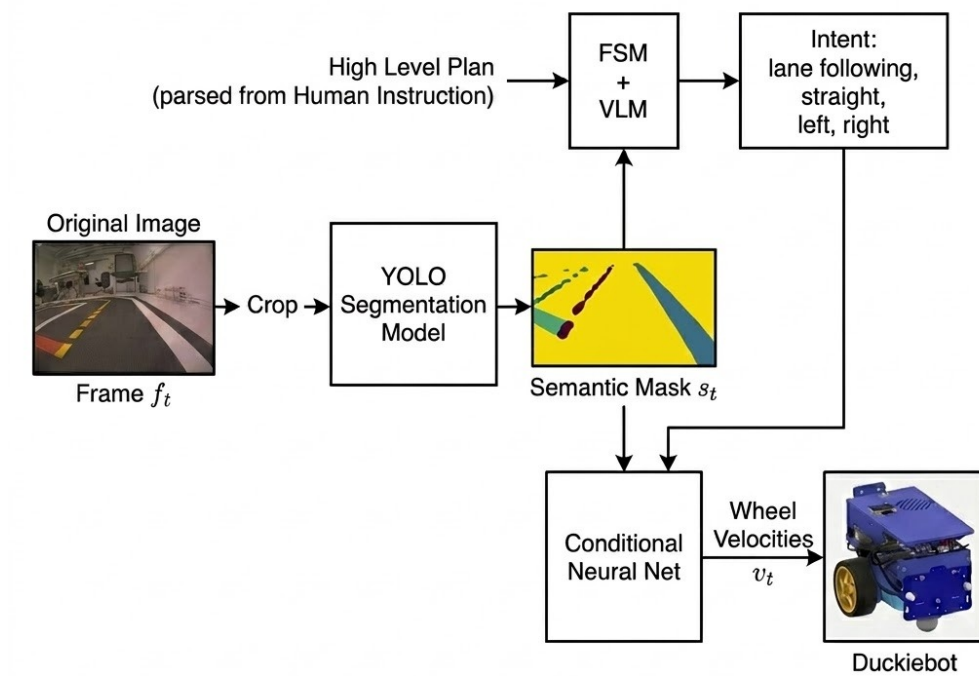


Figure A.1.: Inference pipeline



Figure A.2.: Example output of our trained YOLO segmentation model.

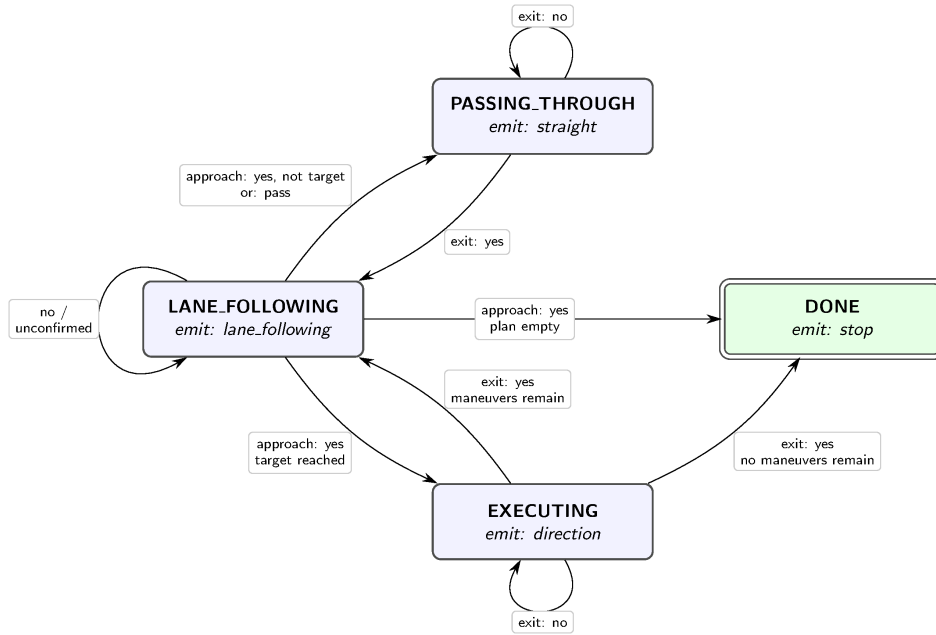
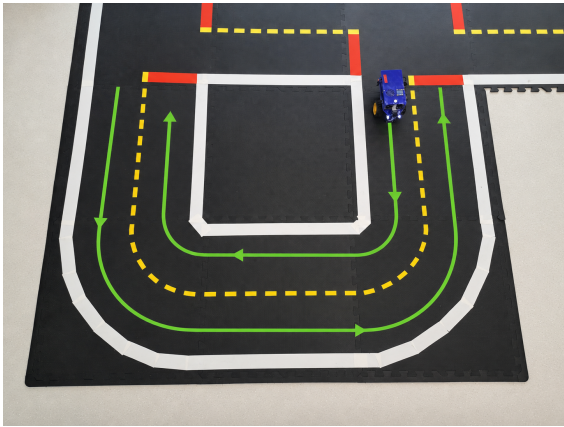
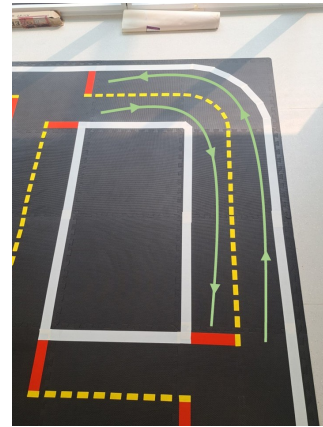


Figure A.3.: The navigation FSM. Each state emits an intent; transitions are driven by the oracle's answer to the FSM's *approach* or *exit* query.

A.2. Task Definitions



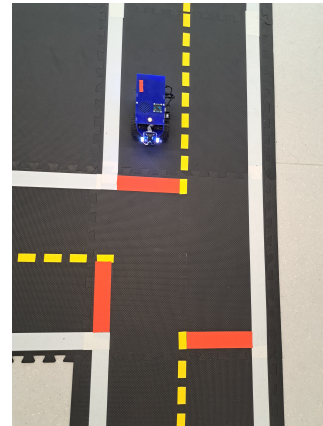
(a) **Semicircle:** "right" refers to the inner track and "left" to the outer loop.



(b) **Long corner:** "right" refers to the inner track and "left" to the outer loop.



(c) **4-way:** Intersection to assess "straight", "left" and "right" trajectories.



(d) **3-way:** Intersection to assess "straight", "left" and "right" trajectories.

Figure A.4.: Tasks from the Experiments Section 3 which test the intents in an isolated manner.



Figure A.5.: Long run: Continuously driving along a predefined loop.

A.3. Network Architectures

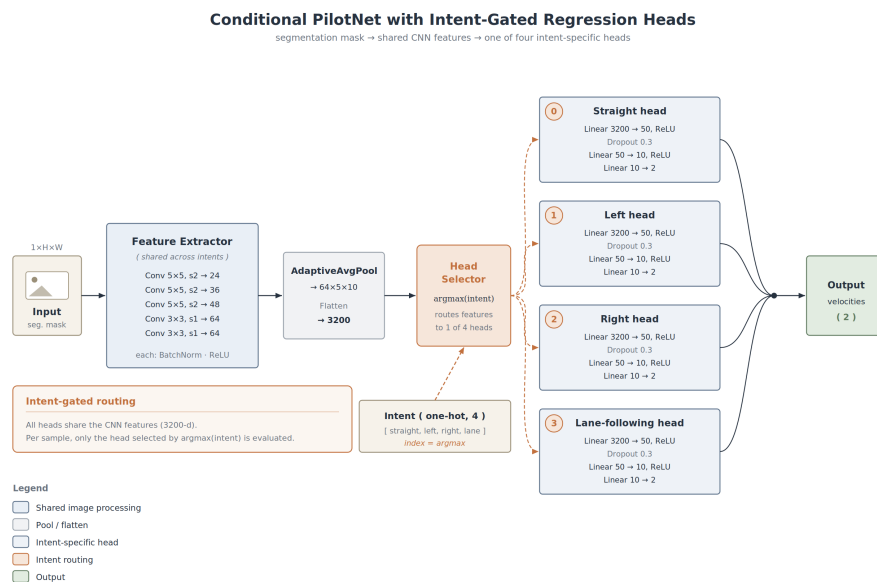


Figure A.6.: Architecture with multiple "heads".

A.4. Other Approaches

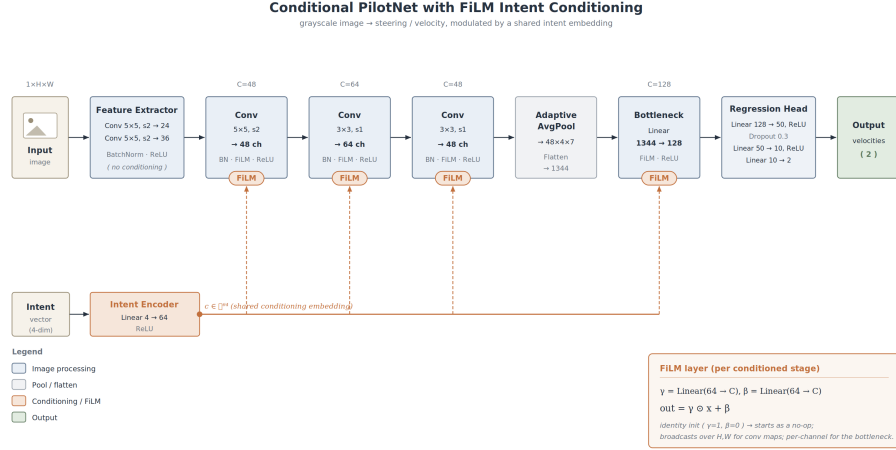


Figure A.7.: FiLM architecture.

A.4. Other Approaches

In this section we briefly describe some other methods that we tried but which either failed or did not produce the desired results.

A.4.1. Naive approach

Initially we attempted to pass the RGB frames directly to the neural network (CNN followed by fully-connected layers). The intent was provided to the fully-connected layers as a one-hot encoded vector. We observed that the model drove forward, but generalized poorly and had issues remaining within the lanes. We hypothesize that the approach likely failed because the model used complete frames as input. It was therefore required to learn both relevant input features and the control policy simultaneously, which is a difficult task when data is limited. Additionally, we believe that the conditioning method was too weak to give the model a sufficient learning signal.

A.4.2. Classification

As described in Section 2.1, we utilized discrete control signals during the data collection phase. Inspired by this, we attempted to train the model to accurately predict appropriate actions given the current input frame. The output layer therefore consisted of 3 neurons, referring to the actions: W = "straight", W + A = "left turn" and W + D = "right turn". We tried this in combination with the multiple head architecture from Section 2.3.1 and observed that the model was able to control the Duckiebot, but also demonstrated a highly non-smooth and zigzaggy driving style. For this reason we decided not to pursue the approach any further.

A.4.3. Time-Shift Regularization

Beyond the architectural and task-based approaches, we experimented with regularization techniques. During deployment, we observed a consistent sensitivity to intent changes, suggesting the learned policies were overfitting to the expert policy in the training data. To address this, we implemented time-shift regularization, which involves shifting intent and velocity labels by a small number of time steps relative to the ground truth. This approach assumes that the expert policy is stochastic; therefore, intents or velocities remain valid across a range of time steps. We also hypothesized that this would increase robustness to latency errors, which accumulate when the model is deployed on a remote machine rather than locally on the Duckiebot.

To test this, we trained the FiLM architecture with various time-shift values and qualitatively evaluated its performance on the Duckiebot. The regularization was implemented as a Gaussian distribution around the ground truth timestamp (illustrated in Figure A.8). Given the 10Hz data collection rate, we tested velocity and intent label shifts at a standard deviation of 0.1s, both independently and in combination. Additionally, we tested a combined shift using a standard deviation of 0.05s, both with and without a 0.05s mean shift into the future.

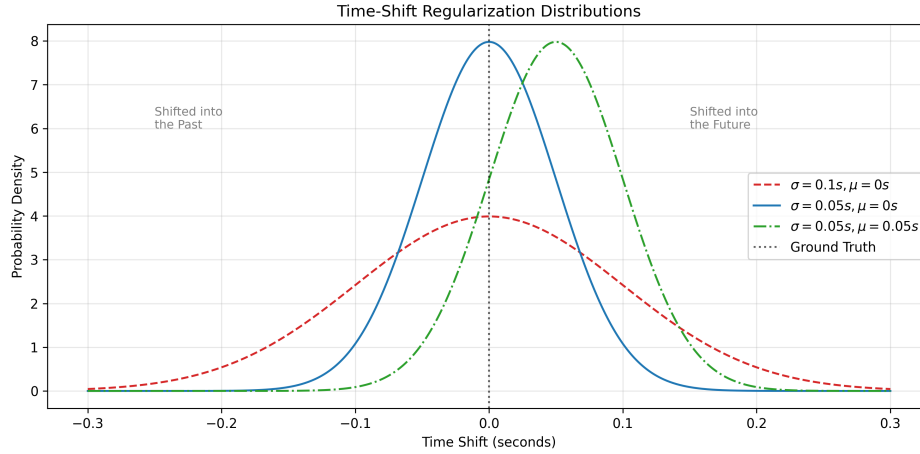


Figure A.8.: **Time-shift regularization:** The figure illustrates the Gaussian distributions used for time-shift regularization.

Results indicated that these models generally underperformed compared to the baseline. Specifically, the Duckiebot frequently failed to maintain its lane and showed no significant improvement in intersection maneuvers. While the models improved at maintaining a straight path at intersections, they still trailed the baseline model trained with the sampling strategy described in Section 3.1. Among the variants, the model with a 0.05s standard deviation and mean shift performed best, whereas all models with a 0.1s standard deviation performed worst. We suspect that shifting labels into the past causes the model to learn a policy that lacks the reactivity required for lane following. Shifting the Gaussian mean into the future reduced

the likelihood of past-label shifts, likely accounting for that model’s superior performance. Given these results, we chose to prioritize other approaches rather than further investigate these temporal shifts.

A.4.4. Sim2Real Transfer

Like the vast majority of deep learning applications, conditional imitation learning is exceedingly data-hungry. Furthermore, the continuous, stochastic, and partially observable characteristics of the environment the Duckiebot must navigate exacerbate this issue. Even with strong assumptions regarding the layout of the Duckietown environment, approximately 11,000 samples were required for the CIL models to reach the performance levels documented in this report. If these findings are to be extended to novel or more complex environments, a question that naturally arises is how to collect the vast amounts of required data in a scalable fashion. To this end, we evaluated the suitability of the Duckiematrix simulation engine.

Our pipeline follows a two-stage approach: the scene is first summarized into a semantic mask, and a policy network subsequently predicts wheel velocities. Consequently, our ablation study focuses on each component separately. We collected two datasets using the same expert policy, one within the Duckiematrix environment (4,350 samples) and the other in real life on the physical Duckiebot (3,750 samples). Both datasets were balanced to contain a similar distribution over intent classes, ensuring that intersection and lane-following scenarios were represented equally.

The first ablation involved investigating the Sim2Real transfer of the segmentation model. We used the SAM3 [CGH⁺26] foundation model to annotate both the virtually and physically collected data. We then fine-tuned YOLO26n [JQL⁺26] semantic segmentation models using the resulting lane-marking masks. Three variants were trained: one on virtual Duckiematrix data, one on solely physical data, and one on a merged dataset. All models were evaluated qualitatively on the physically collected data (Figure A.9). While all models segmented accurately in regions close to the camera, the model trained exclusively on virtual data struggled with classification accuracy at a distance.

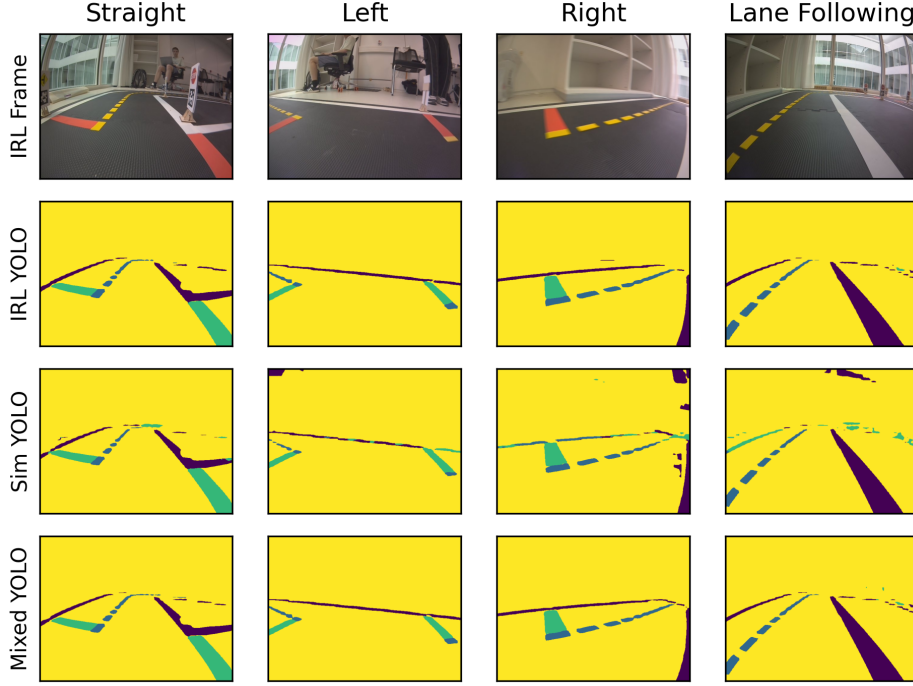
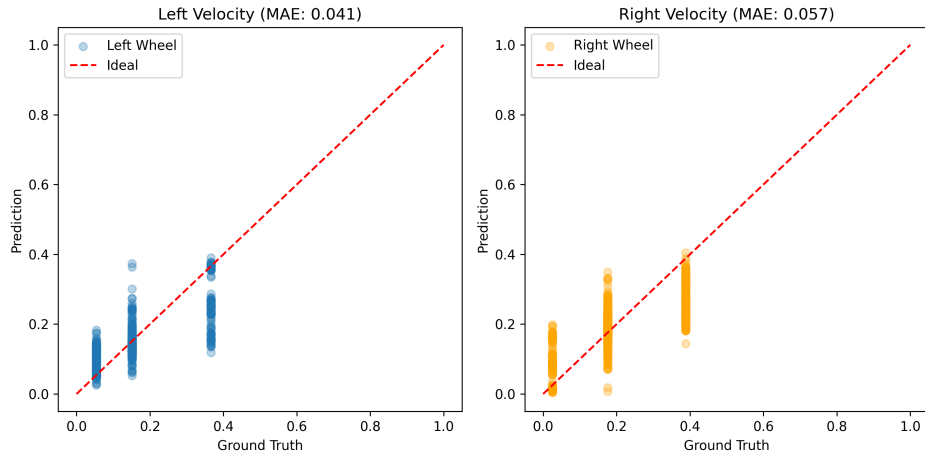


Figure A.9.: **Segmentation Samples:** The figure shows samples from each YOLO26n model trained on either virtual, physical or all data.

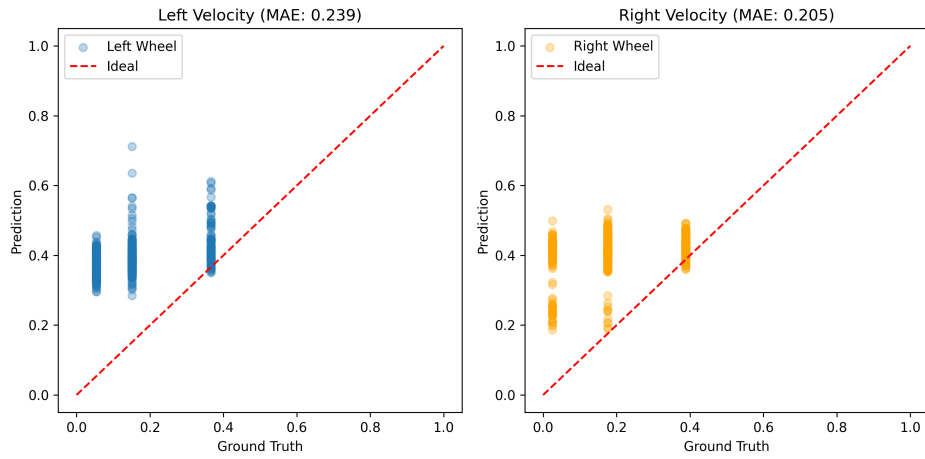
The second ablation addressed the policy network. For all experiments, we used the multi-head architecture depicted in Figure A.6. We trained three variants using virtual, physical, or mixed data, utilizing the corresponding segmentation model from the first ablation for each. We then evaluated their ability to predict the left and right wheel velocities of the physical test data using Mean Absolute Error (MAE), as shown in Figure A.10. The model trained on virtual data consistently overestimates wheel velocities, while the mixed-data model achieves performance comparable to the model trained solely on physical data.

In summary, data from Duckiematrix did not degrade performance, but it also did not yield a significant improvement. Relying purely on simulated data did not prove to be a viable strategy for achieving real-life performance. We hypothesize that this is primarily due to an implementation discrepancy. Despite taking precautions to ensure data collection settings, such as the robot’s overall angular and linear velocity, remained consistent across virtual and physical setups, we observed that velocity targets were consistently higher in the virtual dataset. Furthermore, predicting raw actuator-specific values like wheel velocities may be suboptimal. An alternative approach would be to predict abstract actions pre-mapped to velocity values, similar to the method in Section A.4.2. By coupling this with the segmentation model, the policy network would be more effectively isolated from the environment, potentially facilitating more successful Sim2Real transfer.

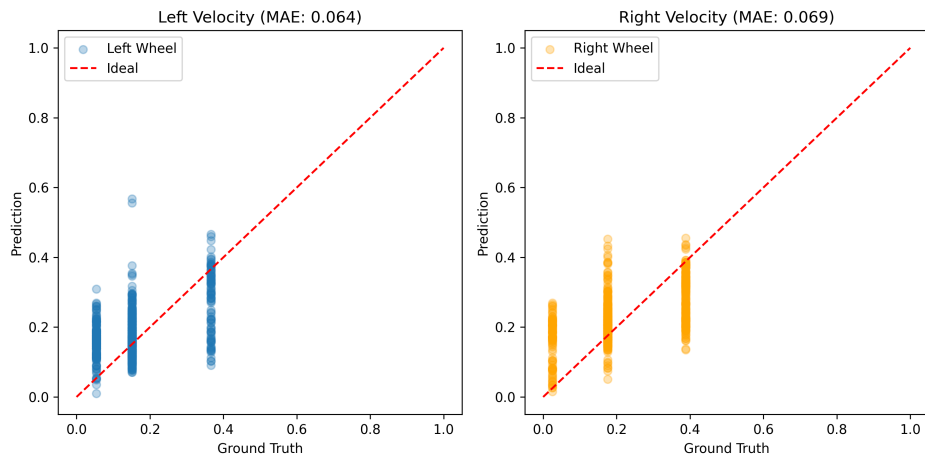
A.4. Other Approaches



(a) IRL2IRL



(b) Sim2IRL



(c) Mixed2IRL

Figure A.10.: **Performance Transfer:** Evaluation across different training policies.

A.5. VLM results

A.5.1. Annotation

We label training data for the oracle with a small web annotation tool built for this purpose. For each frame, an annotator marks it either as *approach* (is a junction visible, and which of left/straight/right are open) or *exit* (has the robot cleared the junction), or skips it. Labels are saved as one JSON entry per frame, keyed by filename.

A.5.2. Prompting

Our zero-shot and few-shot baselines query SmolVLM2 with a fixed, hand-written prompt per question and read off the answer via logit comparison over a closed vocabulary:

Approach: “Is there a road junction ahead, and is {direction} an open path from it? Answer yes (junction, {direction} open), pass (junction, {direction} blocked), or no (no junction). One word only.”

Exit: “Has the robot cleared the junction and returned to normal lane-following? Answer yes or no. One word only.”

For few-shot, we prepend three labeled approach exemplars (one yes, one pass, one no) and two exit exemplars (one yes, one no) as prior turns before the real query.

A.5.3. Fine-tuned Classification Head

Instead of prompting, we freeze the SmolVLM2 vision backbone and train a small multi-label MLP head on top of its pooled features to predict the five targets directly: junction, left/straight/right open, and exit-cleared. We split the image into left/center/right thirds and pool each separately before concatenating, which we found necessary for the model to localize left/right turns rather than only detect a junction. We compare two input types: the cropped camera image (*Photo*) and the cropped, colorized segmentation mask (*Mask*) on SmolVLM2-256M. Since our label distribution is imbalanced, we oversample minority-class frames with augmented copies (flip, jitter) and additionally weight the loss per class. We split by episode rather than by frame to avoid near-duplicate frames leaking between train and validation, and train with Adam and early stopping on validation macro-F1.

A.5.4. CNN Baseline

To check whether the VLM backbone is doing useful work, or whether the segmentation mask alone already contains enough signal, we train a small CNN (four conv layers, the same trunk used by our PilotNet models) with the same multi-label head, labels, split, and oversampling as above, directly on the raw segmentation mask.

A.5.5. Comparison

Table A.1.: F1 scores for the evaluated approaches.

Metric	Zero-shot Photo	Few-shot Photo	SmolVLM2 Mask	SmolVLM2 Photo	CNN Mask
Junction	0.681	0.719	1.000	0.970	0.836
Left Open	0.720	0.417	0.923	1.000	0.720
Straight Open	0.222	0.000	1.000	1.000	0.933
Right Open	0.250	0.000	1.000	1.000	0.609
Exit Cleared	0.000	0.000	0.945	0.963	0.612
Macro F1	0.375	0.227	0.974	0.987	0.742

Zero-shot prompting fails almost completely on exit-cleared ($F1 = 0$): the question needs a notion of the robot’s recent trajectory that a single frame can’t provide. Few-shot exemplars make this worse rather than better, since a handful of in-context images add latency without giving the model a way to localize itself relative to the junction. Fine-tuning closes this gap: both SmolVLM2 variants clear 0.97 macro F1, with Photo slightly ahead of Mask. The CNN-Mask baseline, trained on identical labels and input, only reaches 0.742, so the pretrained VLM features are carrying real signal beyond what the mask alone gives a from-scratch CNN. Despite its slightly lower F1, we deploy the SmolVLM2 Mask head for live inference. The segmentation mask is restricted to our fixed set of classes by construction, while the raw photo can go out-of-distribution whenever objects not seen during training appear in frame.

Contributions

Elia: Contributed to the initial YOLO instance segmentation model and performed ablation studies regarding various regularization techniques and sim2real transfer.

Eric: Collected the dataset, trained the YOLO semantic segmentation model and implemented the FiLM approach.

Felix: Collected the dataset, implemented the training pipeline and trained the multiple head models and the other approaches.

Pankaj: Created the ROS packages and UI for data collection and model inference, trained naive CIL and implemented the VLM-guided planner pipeline (annotation, training, ablations).

All team members contributed to the report and performed the experiments.

Bibliography

- [CGH⁺26] Nicolas Carion, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, Didac Suris Coll-Vinent, Chaitanya Ryali, Kalyan Vasudev Alwala, Haitham Khedr, Andrew Huang, Jie Lei, Tengyu Ma, Baishan Guo, Arpit Kalla, Markus Marks, Joseph Greer, Meng Wang, Peize Sun, Roman Rädle, Triantafyllos Afouras, Effrosyni Mavroudi, Katherine Xu, Tsung-Han Wu, Yu Zhou, Liliane Momeni, RISHI HAZRA, Shuangrui Ding, Sagar Vaze, Francois Porcher, Feng Li, Siyuan Li, Aishwarya Kamath, Ho Kei Cheng, Piotr Dollar, Nikhila Ravi, Kate Saenko, Pengchuan Zhang, and Christoph Feichtenhofer. SAM 3: Segment anything with concepts. In *The Fourteenth International Conference on Learning Representations*, 2026.
- [CML⁺18] Felipe Codevilla, Matthias Müller, Antonio López, Vladlen Koltun, and Alexey Dosovitskiy. End-to-end driving via conditional imitation learning. pages 1–9, 05 2018.
- [JQL⁺26] Glenn Jocher, Jing Qiu, Mengyu Liu, Shuai Lyu, Fatih Cagatay Akyon, and Muhammet Esat Kalfaoglu. Ultralytics yolo26: Unified real-time end-to-end vision models, 2026.
- [MZF⁺25] Andrés Marafioti, Orr Zohar, Miquel Farré, Merve Noyan, Elie Bakouch, Pedro Cuenca, Cyril Zakka, Loubna Ben Allal, Anton Lozhkov, Nouamane Tazi, Vaibhav Srivastav, Joshua Lochner, Hugo Larcher, Mathieu Morlon, Lewis Tunstall, Leandro von Werra, and Thomas Wolf. Smolvlm: Redefining small and efficient multimodal models. *arXiv preprint arXiv:2504.05299*, 2025.

- [PSdV⁺17] Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron C. Courville. Film: Visual reasoning with a general conditioning layer. In *AAAI Conference on Artificial Intelligence*, 2017.